

ASSIGNMENT INSTRUCTIONS

1. This is a **group assignment for CSC 665** (optional CSC 865).
 - Please **form your pair on Canvas by Friday, September 25 at 2pm** by joining a group under *People* $\rightarrow$ *hw2-groups*.
Individual submissions for CSC 665 will not be accepted.
 - You must form a team for this assignment as teams do not carry over from previous assignments.
 - Students who do not have a teammate by the stated deadline will be assigned a partner at random.

Here are the general rules for teams:

- (a) Work on the assignment together. If you divide the work, review, discuss, and combine your solutions into one joint submission.
 - (b) You should **not** just partition the work between you.
 - (c) You should upload only one submission to Canvas.
2. Written Parts: [Problem I except Part 1(a), Problem II Part 1(a), Problem III Part 1]
 - Please type your answers and hand them in as a **PDF** file through Canvas.
 - Your full name(s), as they appear on Canvas, should be included at the top of the PDF.
 3. Coding Parts: [Problem I Part 1(a), Problem II Part 1(b) and Part 2, Problem III Part 2 for extra credit]
 - All source files should be compressed into a *ZIP* archive.
 - Submit the *ZIP* file through Canvas.
 - Be sure your programs execute properly. Failure to do so will mean that your program is not graded.
Note: Submissions are graded from the terminal.
 - File naming format: `firstLastname1_firstLastname2_hw2.zip/pdf` for teams and `<firstLastName>_hw2.zip/pdf` for individual submission (CSC 865 only).
 - **Failure to submit separate PDF and ZIP files on Canvas or follow naming rules will result in an automatic 5-point deduction.**
 - You should use the code provided on Canvas in `homework2_code.zip`
 4. If you use **GenAI**, include a short paragraph stating:
 - which questions you used it for and
 - what changes you made to the output (minor or substantial).
 5. Please refer to the university integrity policy on the syllabus.

NOTE ON GENAI FOR THIS ASSIGNMENT

This assignment involves minimal coding and is meant to assess your understanding of core concepts, your ability to apply them, and your reasoning about the implications of algorithm design choices. Accordingly:

- **Problems I-Part 1(a) (Free use of GenAI with caveat):** For **Part 1 only** you may use GenAI to help generate the implementation, but you are responsible for fully understanding any artifacts you submit.
- **Problems I-Parts 1(b) and Parts 2-3 (Very limited use of GenAI):** Please do not use GenAI tools to generate answers, explanations, or reasoning. You may **proofread your prose** (grammar only), or **ask high level questions that help guide your answers**.
- **Problems II–III (limited GenAI):** You may use GenAI *only* to help translate into Python syntax the formulas you have already derived, clarify error messages, or suggest small refactorings. The logic and modeling choices must be your own. **Understanding the starter code is part of solving the problems, as it provides a concrete example of how logical agents are implemented.**
- **Attribution:** If you use GenAI anywhere, include a brief “GenAI usage” note at the end of your PDF (2–4 sentences) stating *which parts* you used it for and *how* (e.g., “translated my FOL formula to code,” “explained a Python error”), and whether you edited the output.

ASSIGNMENT GOALS AND OBJECTIVES

1. Compare the performance of the adversarial search algorithms `minimax` and `MCTS`.
2. Understand and analyze implementations of logic-based models.
3. Design and implement a propositional logic knowledge base for model checking.
4. Design and implement a first order logic knowledge base for model checking.

PROBLEM I**Monte Carlo Tree Search (MCTS) for Coins in a Line**

This strategy game consists of a line of **n coins**, each with a positive integer value (e.g., [3, 9, 1, 2]). Two players take turns picking coins from either end of the line. On each turn, a player may **pick one or two consecutive coins from either the left or the right end** until there are no coins left.

The goal for each player is to **maximize the total value of the coins** they collect.

Because choices made early in the game affect the remaining options and potential outcomes, players must reason about future moves and anticipate their opponent's actions. In Assignment 1, you implemented **minimax** for this game. In this problem, you will implement **Monte Carlo Tree Search (MCTS)** and experimentally compare it to **minimax** under different computation budgets.

PART 1 – IMPLEMENTATION (MCTS)

Implement an MCTS-based action selector for Coins in a Line (a solution for Assignment 1 is provided in the starter code). You should reuse the existing game logic functions (**actions**, **succ**, **terminal**, **winner/utility**, etc.).

Similar to Assignment 1, there are two main files in this project (more on **benchmark.py** in Part 2): **runner.py** and **coinline.py**. **coinline.py** contains the logic for playing the game, and for making optimal moves. **runner.py** is fully implemented, and contains all of the code to run the graphical interface for the game. However, it will only be functional once you complete the implementation of all the functions in **coinline.py**. You may try your implementation by playing MCTS against the AI (**minimax** in this case). **Please do not modify runner.py.**

- (a) [16 pts] In **coinline.py**, implement the following (*you may use GenAI to help generate this code, but you are responsible for fully understanding your submission*):

1. **terminal_reward**
2. **uct_score**
3. **select_child_uct**
4. **expand**
5. **rollout** (you may use a random rollout policy)
6. **backpropagate**
7. **best_action**
8. **mcts**: returns the next best legal action from **actions(state)**.

Note: the UCT exploration constant, c , is already defined in both **runner.py** and **benchmark.py** as $\sqrt{2}$.

Important: Your successor function **succ** must not mutate the input state. If you modify the state in-place during MCTS, your results may be incorrect.

(b) [4 pts] Include in your PDF submission a brief description explaining the following:

- how you represent nodes and track visits/values,
- how you choose which unexpanded action to expand,
- what rollout policy you used.

PART 2 – EXPERIMENTS AND COMPARISONS

[10 pts] You will compare MCTS against minimax by running multiple games on randomly generated coin lines and recording performance metrics. For that, you will use the additional file `benchmark.py`. You may run the file as it is, please do not modify it. `benchmark.py` is also fully implemented, and contains all of the code to run the automated evaluation.

The following experimental setup is already implemented in `benchmark.py`:

1. Use a fixed line length of $n = 15$ **coins**.
2. Generate N **random coin lines** (choose $N = 30$). Each coin value should be drawn uniformly at random from $\{1, 2, \dots, 10\}$.
3. For each random line, play two games:
 - (a) Game 1: MCTS goes first, minimax goes second.
 - (b) Game 2: minimax goes first, MCTS goes second.
4. Compare **four MCTS budgets per move**: 20, 200, 2000, 20000.

Metrics: For each budget, report at least the following:

1. **Win rate** of MCTS against minimax (percentage of games won).
2. **Average final score difference** (MCTS – minimax).
3. **Average move computation time** for MCTS (e.g., milliseconds per move). (You may also report minimax time if you wish.)

Present your results in a table similar to the following (you may include additional rows/-columns as needed):

Budget	Win Rate	Avg Score Diff	Avg ms/move
20			
200			
2000			
20000			

PART 3 – RESULT INTERPRETATION AND ANALYSIS

1. [6 pts] Write a brief analysis describing trends you observe as the budget increases. Your analysis should address at least:
 - (a) How does performance change as the MCTS budget increases?
 - (b) How does move computation time scale with budget?
2. [6 pts] With the provided starter code, if MCTS is properly implemented, your results may show a very high rate of wins for MCTS.
 - (a) Do you think that is a logical outcome? Justify your answer for full credit. (*remember you are playing AI vs. AI*)
 - (b) Why do you think MCTS has high win rates? (*hint: the results may depend on implementation choices in the provided code*)
 - (c) **[6 pts EC]** Can you make the two AI algorithms have an equivalent performance in terms of win rate? If so, show the result table.
3. [5 pts] Even with large budgets, MCTS may still underperform or show instability in some settings. Name **one reason** MCTS might still struggle on this game, even with a large iteration budget. Your answer must be supported by evidence from your results in Part 2 (e.g., a pattern in win rate, variance across random lines, or sensitivity to budget).

PROBLEM II**Knights and Knaves**

“Knights and Knaves” puzzles are short logic problems about characters who either always tell the truth (Knights) or always lie (Knaves). Each character makes a statement; an agent should deduce everyone’s type from those statements.

Your task is to represent “Knights and Knaves” puzzles using propositional logic, such that a model-checking algorithm can solve them.

- `knights/logic.py`: defines classes for logical connectives (e.g., `And`, `Or`, `Not`) and a `model_check` function that considers all models to test entailment.
- `knights/puzzle.py`: defines propositional symbols like `AKnight` and `AKnave` for characters A, B, and C.
- There are three knowledge bases (`knowledge1–knowledge3`) corresponding to Puzzles 1–3 that you will complete.
- The main function runs model checking to print any conclusions the algorithm can make.

PART 1 – CONNECTIVES

In addition to the five connectives of propositional logic, one occasionally used connective is *exclusive or* (a.k.a. XOR, denoted by $\oplus$): the expression $P \oplus Q$ is true iff exactly one of P or Q is true.

- (a) [5 pts] Write an expression equivalent to $P \oplus Q$ using only $\wedge, \vee, \neg$.
- (b) [10 pts] For this problem to work, it would help to add the XOR connective to `logic.py`. Implement the class for this connective.

PART 2 – KB DEFINITION

Add necessary knowledge to the knowledge bases to enable a logical agent to solve the following puzzles:

1. **Puzzle 1** [6 pts] (single character A): A says, “I am both a knight and a knave.”
2. **Puzzle 2** [6 pts] (characters A and B): A says, “We are the same kind.” B says, “We are different kinds.”
3. **Puzzle 3** [6 pts] (characters A, B, and C):
A says either “I am a knight” or “I am a knave” (unknown which).

B says, ‘A said, “I am a knave.”’ and then says, “C is a knave.”
C says, “A is a knight.”

Once a knowledge base is completed, run `python puzzle.py` to see the solution.

Hints:

- Encode both the puzzle structure (knight $\Rightarrow$ truth-teller, knave $\Rightarrow$ liar, each is exactly one of the two) and what was said.
- Express “if a character says S ” with implications linking `XKnight/XKnave` to S and $\neg S$.
- Aim for a direct, concise translation so the inference (not you) does the reasoning.

PROBLEM III**Odds and evens**

We will use logical methods to automatically prove a theorem by encoding its premises and letting inference do the proving.

Antecedents:

- (a) Each number x has exactly one successor, which is not equal to x .
- (b) Each number is either odd or even, but not both.
- (c) The successor of an even number is odd.
- (d) The successor of an odd number is even.
- (e) For every number x , the successor of x is larger than x .
- (f) “Larger than” is transitive: if x is larger than y and y is larger than z , then x is larger than z .

Consequence: For each number, there is an even number larger than it.

PART 1 – NATURAL LANGUAGE TO FOL

[21 pts – 3 pts each] In your PDF submission, write all six formulas for (a)–(f) and the consequence (query) in First-order Logic.

PART 2 – EXTRA CREDIT IMPLEMENTATION

[6 pts **EC**] In `ints/submission.py`, implement `ints()` to construct the six formulas for (a)–(f). The consequence (query) is provided. Variables must be strings starting with `$`. See `examples.py` for the first-order logic syntax used in this homework.

Autograder: An autograder was provided to evaluate the correctness of your implementation. You can test your code using the following commands:

```
python grader.py 4a-1
python grader.py 4a-2
python grader.py 4a-3
python grader.py 4a-4
python grader.py 4a-5
python grader.py 4a-all    # conjunction of all formulas
```

To prove the theorem, tell the formulas to the knowledge base and ask the query by running model checking (on a finite model):

```
python grader.py 4a-run
```

To run all test cases, use the following command:

```
python grader.py          # run all tests
```